《人工智能编程基础》第十四周上机练习

💬 回顾：你已经完成了什么？

在前两周的练习中，你已经实现了一个**对话系统**，可以在你自己的电脑上运行，向它提问，它就能返回回答。

❓ 现在的问题：如果别人也想用你的对话系统，怎么办？

一种**最简单**的方式是：

- 把你的代码复制一份给别人，让他们在自己的电脑上运行。

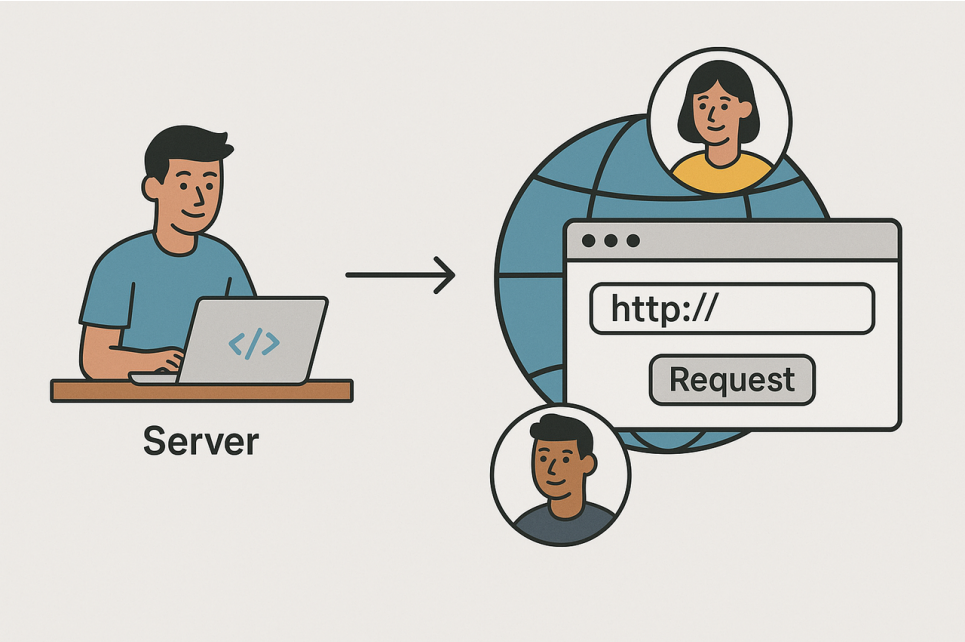
但这样有什么问题呢？

- 一旦你对代码做了修改（比如修复了 bug、添加了新功能），你就得重新发一份给每个人，**非常麻烦**。
- 对方还得自己安装 Python 环境、各种依赖库，**稍不留神就会出错，根本跑不起来**。

🌐 有没有更好的办法？有！那就是 —— 通过网络提供服务

你可以：

1. 在你自己的电脑上**运行一个服务**，这个服务专门接收“提问”，并返回“回答”。
2. 别人只需要通过网络，把问题发给你就行了（比如访问一个网址）。
3. 你的电脑收到请求后，调用你写的对话代码，把答案再返回给对方。



就像你在调用大模型服务时，它也在远程服务器上运行，你通过网络请求向它发送数据，它在远程服务器上执行完后把结果返回给你 —— 你现在要做的，就是把你的对话系统也变成这样！

🎯 本周上机练习

- 把自己写的对话系统**包装成一个Web API 服务**（使用 FastAPI 实现），让其他人可以通过一个 URL 来调用你写的问答功能。具体教程参见以下实验文档。

一、实验目标

- 使用Python的FastAPI库搭建一个简单的Web API服务（注：理解代码框架即可）
- 将现有对话功能改造成Web API接口，加入服务中
- 用户可以通过curl与大语言模型进行对话

二、实验步骤

（一）运行一个简单的Web API服务

这部分我们将运行一个后端服务，并在浏览器中访问 `http://127.0.0.1:8000/` ，看到返回的"hello world"

1. 下载实验代码：请从eLearning平台下载压缩文件 `LLM_helper_backend.zip`
2. 解压并运行服务：打开终端，依次输入以下命令：

代码块

```
# 解压后进入LLM_helper_backend目录
cd LLM_helper_backend
# 根据目录下的requirements.txt中列出的依赖库递归地下载依赖
pip install -r requirements.txt
# 启动服务
python3 main.py
```

看到类似 “*Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)*” 的输出表示后端服务已成功启动

3. 在浏览器中访问 `http://127.0.0.1:8000/` 时就能够看到页面返回"hello world"
   - 如果你和其他同学处在同一个局域网下，他们也可以通过访问 `http://[你电脑的IP地址]:8000/` 来看到页面返回"hello world"。对于你自己来说 `127.0.0.1` 是**环回地址 (Loopback Address)**。它在你的机器上代表**本机 (localhost)本身**
4. 这个Web API服务是怎么实现的呢？

- fastAPI框架帮我们实现了 接收发送到该服务的外部网络请求 以及 将执行结果再发送给请求方 的功能（还有更多，可以查看文档），因此，我们只需要将URL和对应的功能函数对应起来，然后实现具体的功能函数
- 代码简析：在 `main.py` 中，
  - `uvicorn.run(app, host="0.0.0.0", port=8000)` 这个服务会在本机的 8000 端口上对外开放，监听本机的所有IP地址，任何连接到你局域网络的人都可以访问
  - 以下代码表示将 `/` 这个路径和 `root` 函数对应起来：发往 `http://本机任意IP:8000/` 的请求，框架都会自动调用 `root` 函数，然后框架将函数的执行结果再发回给请求方

  注册最简单的路由

  ```python
  @app.get("/")
  def root():
      return "hello world!"
  ```

  > 💡 感兴趣具体的client和服务是如何具体通信的，可以自行阅读HTTP通信协议，里面讲述了client以什么样的规范向服务进行数据传输，还包括了HTTP请求方法的介绍（比如 `@app.get()` 中的 `get` 是一种请求方法）

- 项目结构说明（被*标注的是你可能需要修改的）：

  tree doc

  ```
  .
  ├── utils                 # 该package管理各种工具函数、类
  │   ├── lifespan.py       # 管理启动后端的初始化和退出代码
  │   ├── error.py          # 注册了一些异常处理函数，可以不用管
  │   ├── decoration.py     # API的成功返回格式，不建议修改，若修改需要同步修改前端中的逻辑
  │   └── __init__.py       # 暴露该package下的接口、类、etc.
  ├── services             *# 该package管理你在前两次实验中的代码
  │   └── __init__.py      *# 暴露该package下的接口、类、etc.
  │   └── ...              *# 可以添加其他文件
  ├── requirements.txt     *# 依赖库，如果你需要其他依赖，请添加在这里
  ├── main.py               # 后端入口，在这里注册路由
  └── README.md
  ```

（二）client向服务端传输数据

这一部分你可以请求 `http://127.0.0.1:8000/hello` ，并且附带上自己的姓名（比如，Joe），让服务根据你的输入来返回对应的输出，比如"hello Joe"

1. 使用curl工具发送请求：
   a. curl 是一个在终端中发送 HTTP 请求的命令行工具。你可以用它来“模拟”客户端和服务器之间的通信。
      - 对于Linux和macOS用户来说，curl命令行工具通常是预装的，因此不需要安装
      - 对于Windows用户来说，可能需要手动安装，请参见这个安装教程
   b. 在终端中执行以下命令

```
代码块
curl -X 'POST' 'http://127.0.0.1:8000/hello' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "name": "Joe"
}'
```

      - `-X 'POST'`：表示使用 HTTP 的 POST 方法（用于发送数据）
      - `-H 'accept: application/json'`：表示客户端期望接收 JSON 格式的响应
      - `-H 'Content-Type: application/json'`：表示客户端发送的数据是JSON格式
      - `-d '{ "name": "Joe" }'`：单引号里的内容就是client向服务端发送的数据。你可以尝试将 "Joe" 修改为其他的名字，看看会发生什么
   c. 预期输出：你应该会看到后端根据你提供的名字返回信息：

```
代码块
"hello Joe"
```

2. 除此之外，FastAPI官网提供了更复杂的例子，感兴趣的同学可以自行查阅文档

> 🤖 强烈建议：你可以参考这里，这是FastAPI自动生成的文档和try it out功能。你可以参考，看看它给出的curl命令该怎么写。或者，去问神奇大模型吧！！！

（三）实现对话功能服务

在这个部分里，你需要将之前的上机练习中的对话系统迁移到这个Web服务中，使得其他人可以请求 `http://[你电脑的IP地址]:8000/api/response`，并且附带上自己的问题，就可以得到这个问题的答案

1. 在 `main.py` 中，我们已经将这个对话功能和 `/api/response` 这个路径进行对应了。你们只需要修改 `services` 目录下 `__init__.py` 中的 `answer` 函数，这个函数的参数 `question` 表示的就是用户发送的问题，你们需要在 `answer` 函数中整合之前上机练习的代码，返回对这个 `question` 的回答。（实现过程中，可以在 `service` 文件夹下添加其他文件）
   - 对话功能仅要求上下文感知和外部工具调用，不对流式返回功能进行要求（选做）

代码块

```
# services目录下的__init__.py文件

def answer(question: str) -> str:
    # 你应该在这里想办法调用你前两次实验代码
    # 这个接口将被后端中的路由组件使用
    # 该接口接受一个字符串作为问题，返回一个字符串作为回答
    return "The answer to this question is..."
```

2. 启动这个后端，并使用curl测试它

代码块

```
> python3 main.py
INFO:     Started server process [8436]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
...
> curl -X 'POST' \
  'http://127.0.0.1:8000/api/response' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "question": "你好"
}'
> {"code":0,"data":{"answer":"你好呀！有什么可以帮你的吗？😊"}}
```

3. **（选做，实现可【少量】加分，不实现【不扣分】）**请求 `http://127.0.0.1:8000/api/response` 时，输出为流式返回。

三、提交说明！！！

- 在elearning上提交一个**zip**文件，命名为【学号-姓名.zip】
  - 注：要求zip文件解压缩后即得到修改 `LLM_helper_backend` 仓库

```
代码包LM_helper_backend
├── ...
└── ...
```

- 请务必按照上述格式提交，否则予以扣分

- **要求使用的API Key有效期需要持续到本学期该课程结束**
- 输出格式参考实验步骤（三）.2
- **请在下课前提交一个版本**；在 **2025年5月29日23:59** 前提交终版，在这期间可以多次提交
  - **如果没有提交最终版本，则按照上机课后提交的版本给分**